MiCA Compliance Copilot — Documentation

Course: AI for Developers: Design, Build, Deploy LLM-powered Applications (AUEB). **Type:** Generative-AI application — *not a chatbot*: AI logic embedded in a clean software architecture (FastAPI backend + functional UI + GenAI layer).

Live demo: https://mica.exadaktylos.xyz · **GitHub:** https://github.com/exas-3/mica-copilot (public, MIT).

---

1. Title, description, purpose

MiCA Compliance Copilot is a Retrieval-Augmented + agentic assistant for the EU **Markets in Crypto-Assets Regulation** (Regulation (EU) 2023/1114, "MiCA").

Purpose. Answer MiCA questions *grounded in the actual regulation* with article-level citations, classify a token/service under MiCA, and look up real ESMA register data — while **refusing to answer** when the indexed corpus doesn't support a grounded answer. In a compliance context, a confident-but-wrong answer is worse than "I don't know," so retrieval + citation + abstention are the whole point.

2. Use scenario & functional requirements

Scenario. A compliance analyst or crypto founder asks, *"What backs the reserve of an asset-referenced token?"* or *"Is my euro stablecoin an EMT, and what must I do?"* They need an answer they can trust and trace back to a specific article — not a plausible paraphrase.

Functional requirements

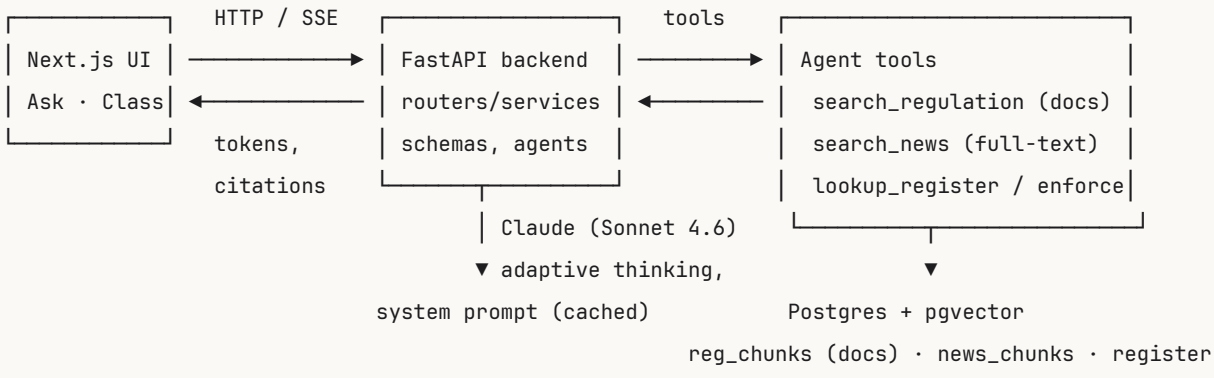
- F1. Answer MiCA questions grounded in retrieved regulation text, with per-article citations.
- F2. Abstain (don't fabricate) when retrieval doesn't support an answer.
- F3. Classify a described token/service: asset type (ART / EMT / other / out-of-scope), the crypto-asset services (a–j) involved, and the resulting obligations — as structured data.
- F4. Look up whether a named entity is an authorised CASP / EMT issuer, or is flagged.
- F5. Stream answers and show the agent's tool-use trace; expose Swagger docs.

3. Technologies used & rationale

Layer	Choice	Why
Backend	**FastAPI**	Async, typed (pydantic) request/response, automatic Swagger/OpenAPI — required by the brief and a clean orchestration point.
LLM	**Claude Sonnet 4.6** (agent loop, default) + **Haiku 4.5** (cheap tasks/judge)	Sonnet balances tool-use reasoning and cost; `AGENT_MODEL=claude-opus-4-8` is a one-line upgrade for maximum quality. Adaptive thinking + `effort`.
Vector store	**Postgres 16 + pgvector**	One self-contained DB (`docker compose up`) holds both the embeddings and the register snapshot; reproducible for a grader.
Embeddings	**mxbai-embed-large-v1** (local, default) / **Voyage** `voyage-law-2`	local model is 1024-dim, ~0.64 GB, runs **offline with no API key** (reproducible key-free); `voyage-law-2` is the legal-tuned paid alternative.
Sources	**EUR-Lex + ESMA/EBA** (docs) · **RSS** (news) · **ESMA register CSVs** (registers)	`feedparser` + `trafilatura` + `pypdf` ingest the regulation, RTS/ITS, guidelines/Q&As, full news articles, and the real ESMA registers (incl. Title II white papers, whose token is read from each document).
UI	**Next.js 15 / React 19**	Streaming chat + structured result rendering; design system adapted from the author's `mica-dashboard` (warm-paper + EU-blue, hand-rolled — no Tailwind).

> **Provenance & method.** For the full source inventory (every document, feed and register, with licensing + refresh cadence) see `DATA-SOURCES.md`; for the end-to-end pipeline (chunking, embeddings, retrieval, routing, evaluation, with exact parameters) see `METHODOLOGY.md`.

4. Architecture & data flow



Request flow for `/chat`: 1. UI POSTs the question; backend opens a streamed agent loop. 2. Claude routes: regulatory questions → `search_regulation` (embed → match `reg_chunks`); current-status questions → `search_news` (recency-ranked `news_chunks`) **+** `lookup_register` / `check_enforcement`. 3. Claude answers **only from retrieved sources** — citing documents by article/instrument and news by source + date — and abstains otherwise; the backend streams `token` / `tool` / `citations` / `done`.

Ingestion. `app.rag.docs_ingest` fetches the regulation + RTS/ITS (EUR-Lex) and ESMA/EBA guidelines/Q&As (PDF/HTML) → article/section chunks → `reg_chunks` (+ register seed). `app.news.poll`

polls 14 RSS feeds, fetches **full article text** (`trafilatura`), triages relevance (Haiku), and embeds into `news_chunks` (recency-ranked). Both idempotent; the document corpus rebuilds offline from a committed cache.

Privacy & telemetry. Two telemetry paths, both privacy-minimising: (1) **conversation logging** — each question + answer is written to a write-only `chat_logs` table for quality/debugging; it is **not** exposed to the agent (no tool or retrieval path reads it) and carries no identifier; and (2) **analytics** — privacy-friendly, **cookieless** Plausible, **self-hosted** on our own server and **proxied first-party** through `/pa/*` (a Next.js rewrite — avoids `http→https` mixed content and keeps the browser on our own domain). No cookies, no cross-site tracking, no third-party analytics provider. See the Privacy Policy.

Deployment. Only the **Next.js UI (:3000)** is internet-facing — it is exposed through a **Cloudflare Tunnel** (`cloudflared` , systemd `mica-tunnel` service), so there are no open inbound ports. The browser calls `/api/*` **same-origin**, and Next.js (`next.config.mjs` rewrites) forwards those **server-side** to the **localhost-only FastAPI backend** (`127.0.0.1:8000` , systemd `mica-api`) — so the API is never internet-facing and no CORS is needed. Self-hosted **Plausible** analytics are proxied first-party via `/pa/*` (avoids `http→https` mixed content and ad-blockers). **Postgres + pgvector** runs in **Docker**. All services use `Restart=always` + user lingering, and the frontend runs a **production build** (`npm run build && npm start`), not `next dev` . See `deploy/TUNNEL.md` for the full deployment architecture.

5. FastAPI endpoints & services

Routers (`app/routers/`): `chat` (`/chat` SSE, `/chat/sync`), `classify` (`/classify`), `registry` (`/registry/search`), `health` (`/health`). All inputs/outputs are pydantic schemas (`app/schemas`), giving validation + the Swagger contract.

Services (`app/services/`): - `llm.py` — the Claude client, the **manual agentic tool loop**, **SSE streaming**, **prompt caching**, and **structured classification**. - `rag.py` — embed query → vector search → build the citable context block → dedup citations. - `registry.py` — read-only register lookups (degrade gracefully if a table is empty).

Agent layer (`app/agents/`): `tools.py` (tool definitions + local dispatcher), `prompts.py` (the cached system prompt + MiCA map + the classification JSON schema).

RAG layer (`app/rag/`): `embed.py` (Voyage/local behind one interface), `chunk.py` (article-aware chunking), `store.py` (pgvector upsert/search), `docs_ingest.py` (document corpus), `eurlex.py` (any-CELEX fetch/parse), `pdf.py` (PDF/HTML extraction).

News layer (`app/news/`): `sources.py` (14 RSS feeds), `poller.py` (conditional-GET, dedup, full-text fetch, classify, embed), `classify.py` (Haiku triage), `entities.py` (entity match), `store.py` (recency-weighted `news_chunks` search), `poll.py` / `scheduler.py` (CLI / APScheduler).

Register layer (`app/register/`): `sync.py` syncs the **real** ESMA register CSVs (CASPs, EMT/ART issuers, ~877 Title II white papers, warnings — `parse.py` handles dates/arrays/dedupe), and `whitepapers.py` **reads each white-paper document** (PDF/HTML) with Haiku to extract its token name + ticker (ESMA's register has no token-name field). `services/registry.py` then searches all of these by firm OR token, so "does Cardano/ADA have a MiCA white paper?" is answered from real data with the white-paper URL.

6. UI & interaction

- **Ask MiCA** (`/`): a streaming chat. Tool calls appear as chips ("Searched MiCA corpus …", "Searched news …"), the answer streams token-by-token **as Markdown** (bold, lists, tables, code and links are rendered as formatted HTML), and a **Citations** panel groups sources into **Regulation & documents** (article/instrument + EUR-Lex/PDF link) and **News** (source + date + link). If the model abstains, a "No grounding — answer withheld" badge shows. A **transparency note** under the chat input states that answers are AI-generated and **not legal advice**, that queries go to Anthropic's Claude API, and that no confidential data should be entered.
- **Classify** (`/classify`): paste a token/service description → a structured card with the asset type, the implied a–j services, the obligations (each tied to an article), and citations.
- **Guides** (`/guides`): four cited MiCA explainers — asset-referenced tokens (ART), EMT vs. stablecoin, CASP authorisation, and white-paper requirements.
- **Docs** (`/docs`): an in-app documentation reader.
- **Privacy** (`/privacy`) and **Terms** (`/terms`): the privacy policy and terms of use.

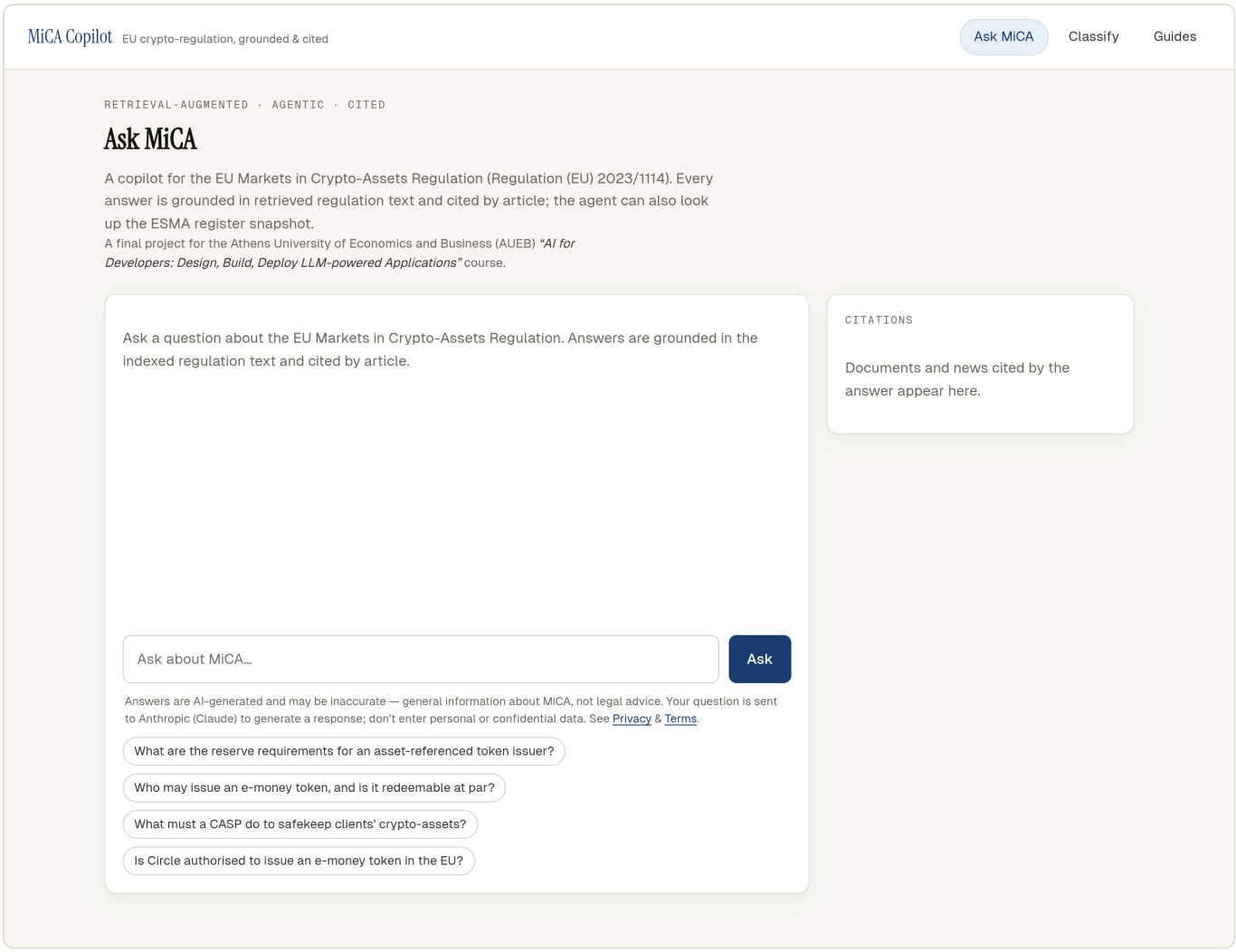
A global **footer** on every page links Guides / Docs / Privacy / Terms / GitHub, carries the AUEB attribution, and repeats the "not legal advice" disclaimer.

The UI is evaluated on functionality, clarity, and ease of use; example prompts are provided so a reviewer can exercise every path in one click.

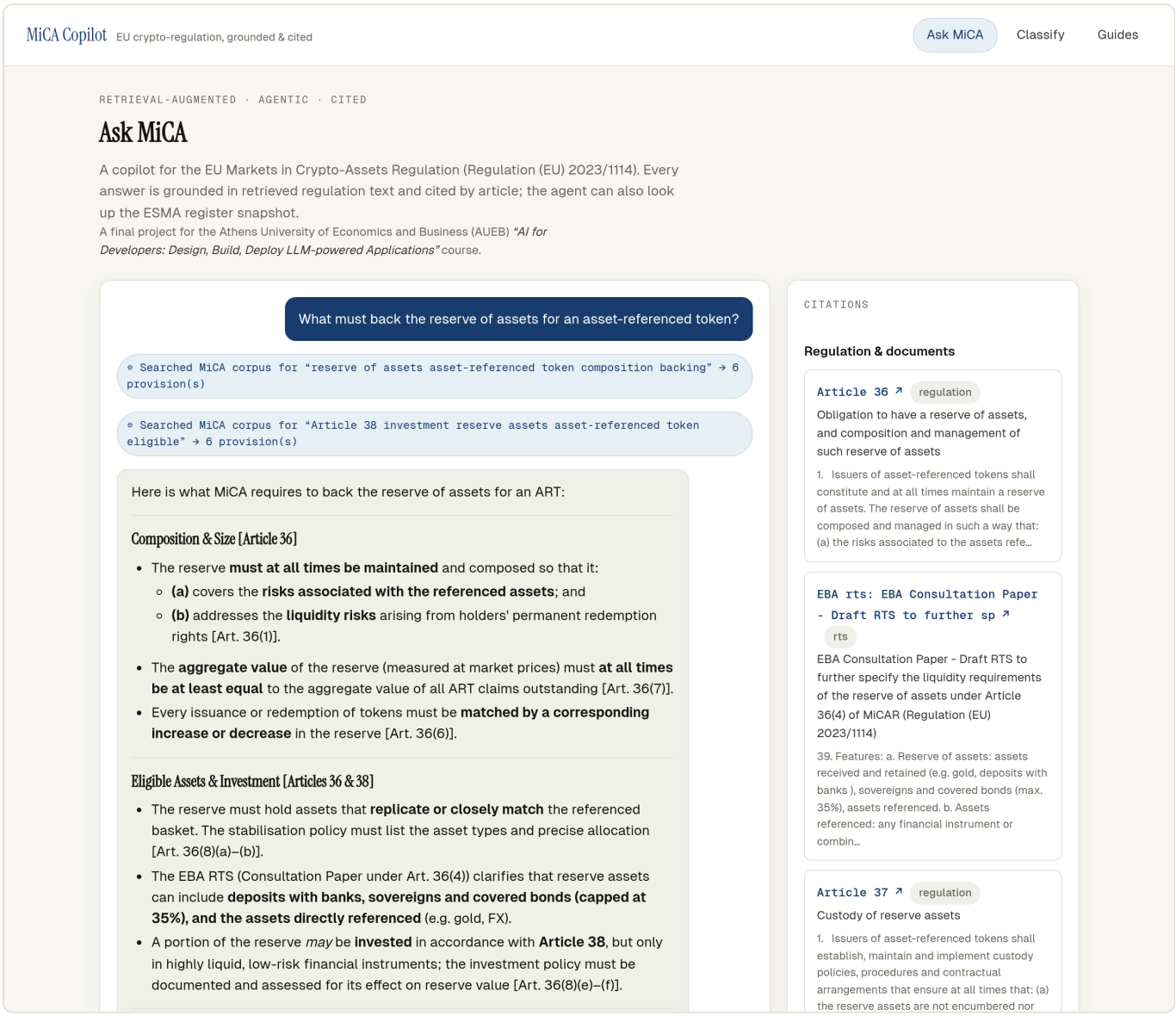
Screenshots

> Screenshots are from the current production UI at mica.exadaktylos.xyz.

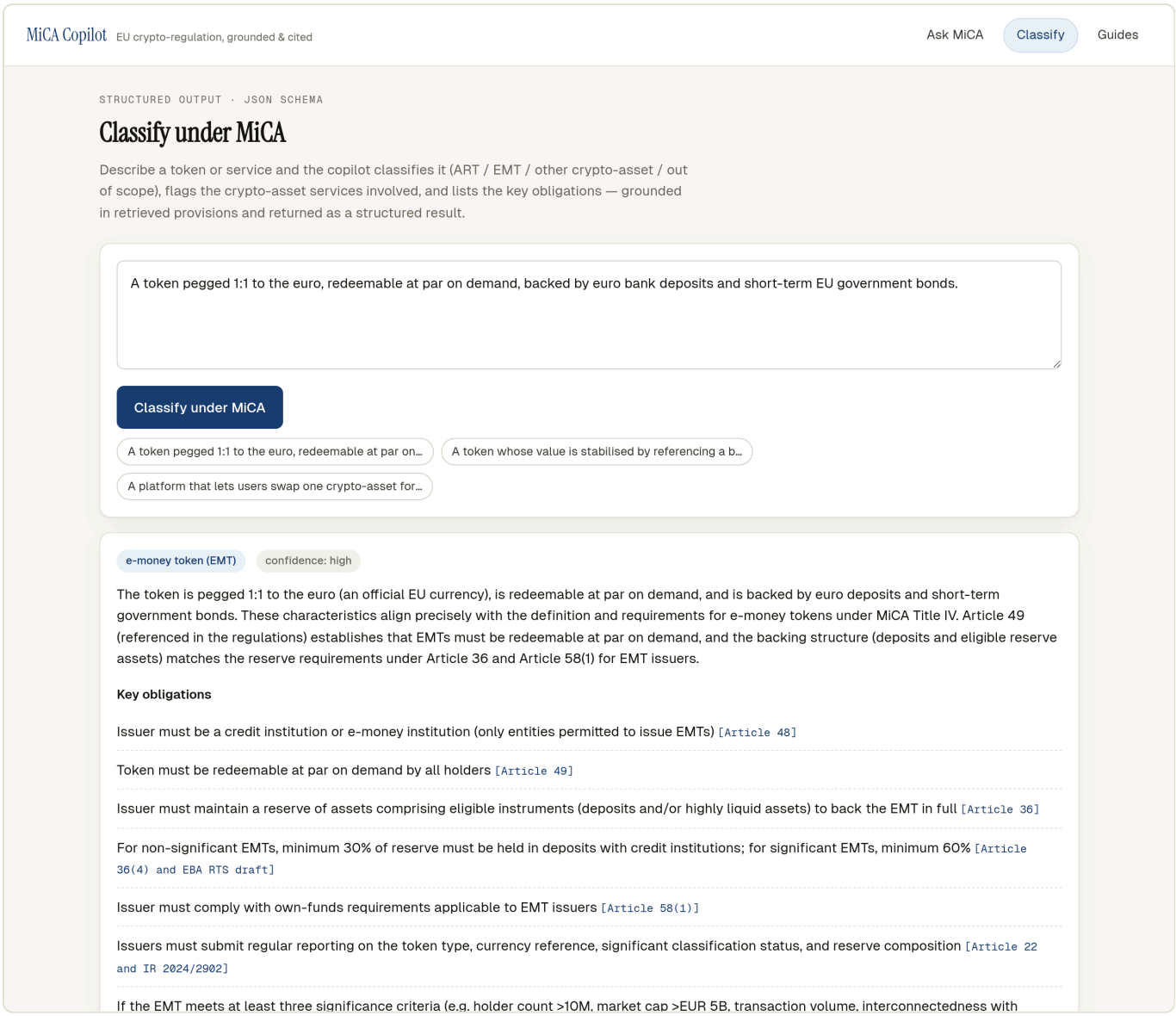
Ask MiCA — landing. Clean entry point with example prompts and the Citations panel (warm-paper + EU-blue design system, no Tailwind).



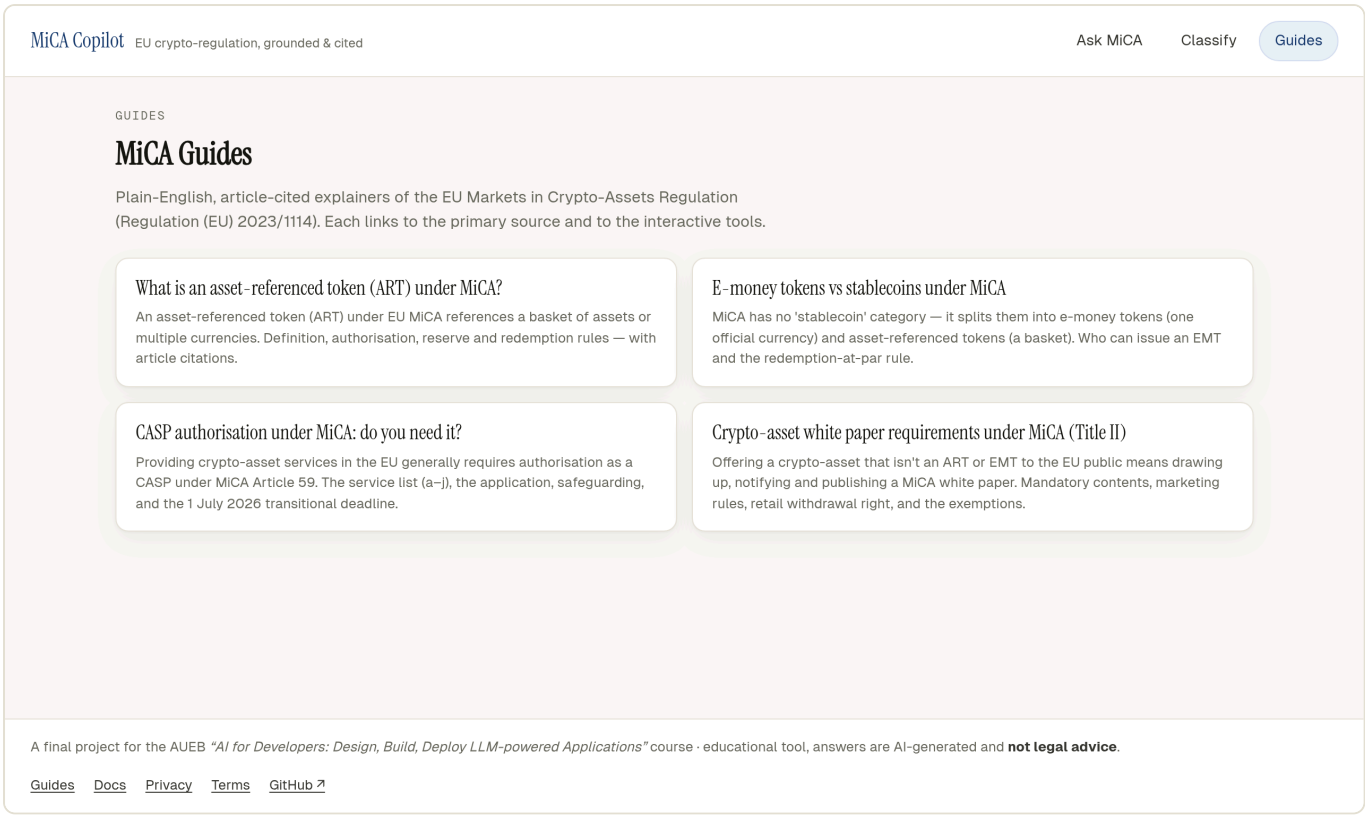
Ask MiCA — a grounded answer. The agent's tool calls show as chips, the answer streams in, and the Citations panel lists the cited provisions (article/instrument + link).



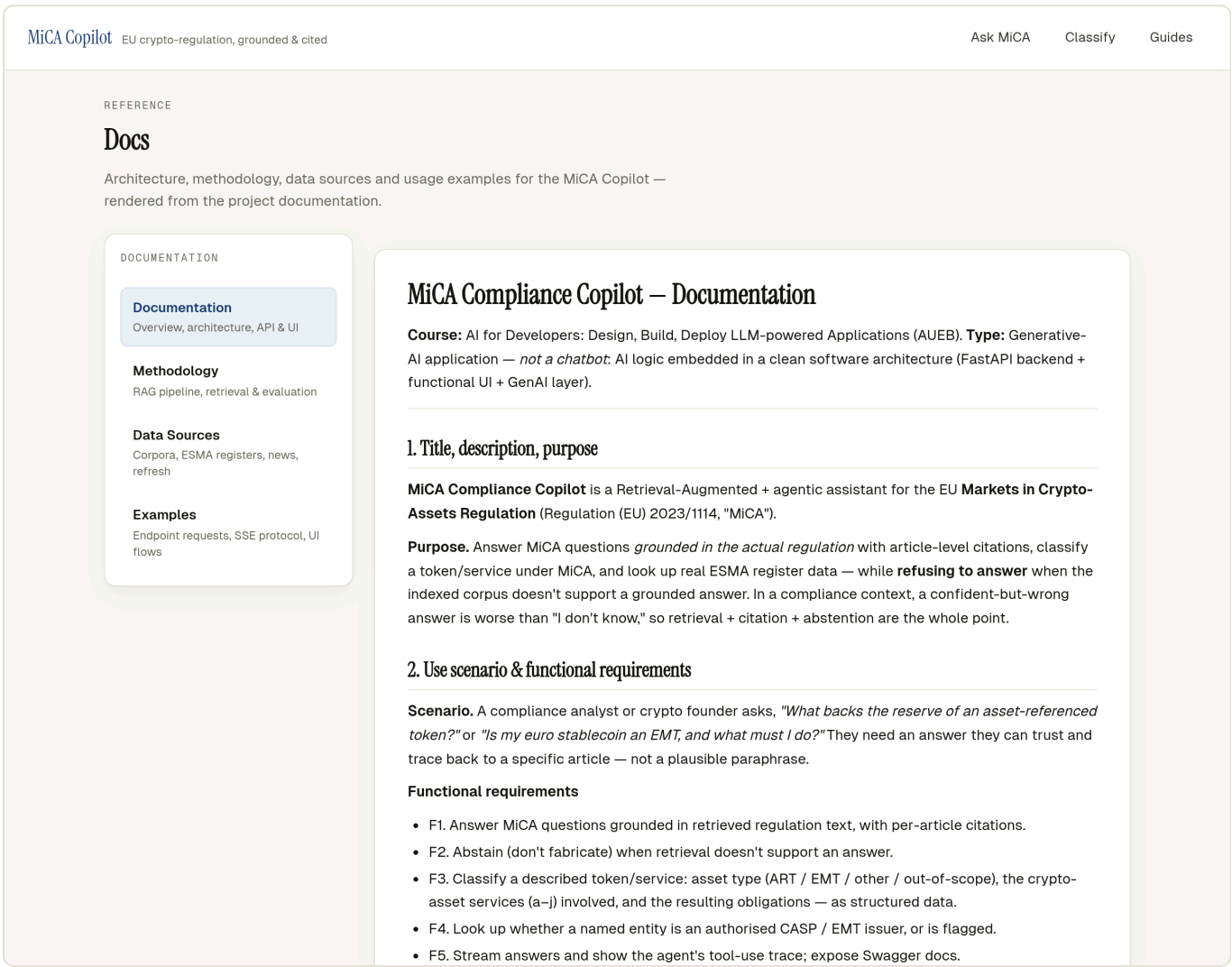
Classify — structured result. A euro stablecoin is classified as an **e-money token (EMT)** with the implied services, the key obligations (each tied to an article), and citations — rendered from the JSON-schema-constrained `/classify` response.



Guides — plain-English explainers. Four cited MiCA explainer pages (ART, EMT vs. stablecoin, CASP authorisation, white-paper requirements), each linking to the primary source and back to the tools.



In-app documentation (`/docs`). This project documentation rendered inside the app with a sidebar.



7. GenAI techniques (detail)

- **Multi-source RAG** — two pgvector corpora: a **document corpus** (`reg_chunks`: the Regulation + RTS/ITS + ESMA/EBA guidelines/Q&As, article/section chunked) and a **full-text news corpus** (`news_chunks`, recency-weighted). Answers must cite retrieved sources; the prompt forbids answering from general knowledge and abstains when support is missing.
- **Tool / function calling + routing** — the agent picks `search_regulation` (what the law requires), `search_news` (current facts, dated), `lookup_register`, or `check_enforcement`; each tool's description states *when* to call it. News is flagged time-sensitive and cited with its date.
- **Structured outputs** — `/classify` constrains the response to a JSON schema (`output_config.format`), so the UI always renders valid structured data.
- **Prompt engineering / system prompts** — a role-based prompt ships a stable MiCA "map" and an explicit abstention rule.
- **Prompt caching** — the large stable system prefix carries `cache_control`, so repeated requests pay cache-read (~0.1×) instead of full price. *Verified:* a repeat request read ~1,693 tokens from cache (`cache_read_input_tokens`) on the second call.
- **Markdown rendering** — assistant answers are formatted markdown (lists, tables, code, links), not plain text.

8. Evaluation results

Run `python -m eval.run --e2e --judge` (golden set in `eval/goldens.jsonl`). Metrics:

Metric	What it measures
`retrieval_hit@k`	Did the expected article appear among retrieved provisions?
`citation_hit`	Did the agent's answer actually cite the expected article?
`abstention_accuracy`	Did the agent decline on out-of-corpus questions?
`faithfulness`	LLM-as-judge: is every claim supported by retrieved context?

Measured run (**44 goldens** — 35 answerable, 6 abstain, 3 register; agent = Sonnet 4.6 at `effort=low`, judge = Haiku 4.5, embedder = local `mxbai-embed-large-v1`; v2 corpus = **1,422 regulation chunks + 63 news chunks**, 1,258 register rows):

Metric	Score
retrieval_hit@k	**0.971**
citation_hit	**0.914**
abstention_accuracy	**1.000**
faithfulness	**0.886**
register_hit	**1.000**

The golden set was expanded from 29 to **44** to cover more of the Regulation — Title III ART authorisation (Art 16), Title IV EMT white paper (Art 51), Title V CASP authorisation / prudential / governance / outsourcing (Art 60, 62, 67, 68, 73) and the per-service rules (Art 76–81), plus the **Art 143 transitional period** — with extra

out-of-scope (*abstain*) and *register* cases. The winning retrieval config (vector + mxbai query-prefix + Article-3 per-definition chunking + `also_accept` credit for the genuine Level-2 elaboration) reaches **retrieval_hit@k 0.971**; the one miss is Art 88, whose base provision ranks below the Level-2 ESMA market-abuse guidelines. **Faithfulness rose to 0.886** after the judge context was made *citation-aware* (it now also sees the exact provisions the answer cited, not just generic top-k retrieval — fixing a measurement artifact, not the answers). Abstention and the ESMA register lookup stay perfect on the larger set, including the new VAT/DORA out-of-scope questions and the MegaETH white-paper lookup. `citation_hit` (0.914) carries run-to-run variance from the stochastic agent — two of this run's three misses are goldens that pass on other runs (the agent sometimes cites a neighbouring article). Reproduce with `python -m eval.run --e2e --judge`; per-question detail in `eval/results/scorecard_improved.json`.

Web performance & accessibility (Lighthouse)

Measured on the live production build (`mica.exadaktylos.xyz`):

Category	Desktop	Mobile
Performance	**100**	**98**
Accessibility	**100**	**100**
Best Practices	**100**	**100**
SEO	**100**	**100**

Core Web Vitals (mobile): **LCP 2.3 s · CLS 0 · TBT 30 ms**. The production `next build` + the same-origin `/api` proxy keep LCP low and layout shift at zero; accessibility reached 100 after making in-text links distinguishable by underline (not colour alone).

9. Limitations & future extensions

Limitations. The document corpus is real full text of public EU sources (regulation + RTS/ITS + ESMA/EBA guidelines/Q&As), rebuildable offline from the committed cache or `docs_ingest --refresh`. The news corpus reflects whatever the feeds carried at the last poll — keep it fresh with the scheduler; trade-press full text is stored for private retrieval and always cited + linked with its date (`NEWS_STORE=summary` switches to transformative summaries). The ESMA registers are the **real** public data synced from ESMA's CSVs; Title II white papers have no token-name field, so each token is read from the white-paper document (best-effort — some links are dead/unreadable, so token-name coverage is partial). General information, **not legal advice**.

Already deployed. FastAPI runs as the systemd `mica-api` service (localhost-only `:8000`), Next.js as `mica-web` (production build, `:3000`), and Postgres in Docker, all exposed via a Cloudflare Tunnel (`mica-tunnel`) at https://mica.exadaktylos.xyz — only the frontend is internet-facing.

Future work. (a) Hybrid retrieval (pgvector + the shipped `tsvector` column) and a Voyage reranker. (b) Multilingual (EL/EN) querying (`intfloat/multilingual-e5-large`). (c) A citation-verification self-check pass.

10. Install / run / examples

See `../README.md` §2–6 for **local / development** install/run, and `EXAMPLES.md` for a full set of copy-paste usage examples (every endpoint with request **and** response, the SSE event stream, the UI flows, and the eval

run).

Production deployment. See `deploy/TUNNEL.md` — the public deployment exposes only the Next.js UI through a **Cloudflare Tunnel** (`mica-tunnel`), with the three systemd services (`mica-api` localhost-only `:8000`, `mica-web` production build `:3000`, `mica-tunnel`), the same-origin `/api` proxy, and the self-hosted Plausible `/pa/*` proxy. The public deployment is live at https://mica.exadaktylos.xyz.

Quick check after setup:

```
curl -s localhost:8000/health | jq          # chunks_indexed (docs), news_indexed, register_rows > 0
curl -s localhost:8000/chat/sync -H 'content-type: application/json' \
  -d '{"message":"Must an e-money token be redeemable at par?"}' | jq '.answer, .citations[].article_ref'
# Level-2 (RTS) + current-facts (news) routing:
curl -s localhost:8000/chat/sync -H 'content-type: application/json' \
  -d '{"message":"Which Delegated Regulation sets the complaints-handling RTS for CASPs?"}' | jq '.citation
curl -s localhost:8000/chat/sync -H 'content-type: application/json' \
  -d '{"message":"What is going on with Binance and MiCA, and is there a deadline?"}' | jq '.tool_events[].
curl -s localhost:8000/classify -H 'content-type: application/json' \
  -d '{"description":"A token pegged 1:1 to the euro, redeemable at par, backed by bank deposits."}' | jq '
```

11. Deliverables mapping (assignment checklist)

How this project satisfies each requirement of the *AI for Developers: Design, Build, Deploy LLM-powered Applications* brief:

Requirement	Where it lives
Not a chatbot — AI logic inside a real software architecture	FastAPI backend (routers / services / schemas / agents) + Next.js UI; the LLM is one orchestrated layer, not the whole app (§4–§5).
Functional UI	`frontend/` — Ask MiCA (streaming chat + citations) and Classify (structured card); §6.
Backend API	FastAPI with auto-generated Swagger at `/docs`; endpoints in §5 and `EXAMPLES.md`.
RAG (required)	Multi-source: `reg_chunks` (official documents) + `news_chunks` (full-text news), pgvector cosine search, article/source-cited answers, abstention (§7).
Agents / tool-calling (rewarded)	Manual agentic loop routing `search_regulation` / `search_news` / `lookup_register` / `check_enforcement` (`app/services/llm.py`, `app/agents/tools.py`).
Structured outputs (rewarded)	`/classify` constrained to a JSON schema via `output_config.format` (§7).
Prompt engineering + caching	Role-based system prompt with an explicit abstention rule; `cache_control` on the stable prefix (verified cache reads, §7).
Evaluation	`eval/` golden set + harness; scorecard in §8 (`eval/results/scorecard.json`).
GitHub repo	This repository.
Documentation (DOC/PDF)	This file (`docs/DOCUMENTATION.md`) — export to PDF for submission.
README	`../README.md`.
Usage examples	`EXAMPLES.md`.
Demo (optional)	Live public UI at https://mica.exadaktylos.xyz (local dev at `localhost:3000`); `EXAMPLES.md` lists one-click example prompts per path.